GreyNOC
Certified Pentest / Ethical Hacking · Michigan

Bug Bounty Hunting Playbooks

Repeatable, scope-disciplined methodology for authorized vulnerability research across web, API, and cloud-facing attack surface.

DOCUMENT	**GN-PUB-BBHP-001**
VERSION	**v1.0.0**
CLASSIFICATION	**PUBLIC // GreyNOC Field Reference**
TEMPLATE	**GN-TPL-PUB-001**
DISCIPLINE	**Authorized testing only · Reproducible findings · No fabrication**

FRONT MATTER

Authorization, Scope & Use

These playbooks document defensive and offensive methodology for **authorized vulnerability research only** — specifically, participation in bug bounty and vulnerability-disclosure programs (VDPs) where the program owner has published an explicit grant of permission to test in-scope assets. They are GreyNOC field reference material and assume the operator already holds the legal right to test the target before any technique in this document is applied.

OPERATING CONSTRAINT — READ BEFORE TESTING

Every technique below is gated on a written, in-force authorization for the specific asset under test. A public bug bounty page *is* that authorization, but only within the exact scope, asset list, and rules of engagement (ROE) it publishes. Testing outside that boundary is not research — it is unauthorized access, and nothing in this document licenses it.

When scope is ambiguous, the default is STOP and ask the program, not proceed.

How to use these playbooks

Each playbook is a self-contained, repeatable procedure for one vulnerability class. They share a fixed structure so findings stay reproducible and reports stay consistent:

- **Objective** — the bug class and the impact you are trying to demonstrate.
- **Scope precondition** — what must be true in the program scope before you run it.
- **Methodology** — ordered steps from signal to confirmed, evidenced finding.
- **Tooling** — representative tools; substitute equivalents freely.
- **Signals** — indicators that the class is present and worth deeper testing.
- **Evidence & PoC** — the minimum proof a triager needs to accept the report.
- **Pitfalls / false positives** — common ways researchers waste a submission.
- **Reporting notes** — severity framing and what to include in the writeup.

GREYNOC STANDARD

No fabrication. Every claim in a report must be reproducible from the steps and artifacts you provide. If you cannot reproduce it twice from a clean session, it is not a finding yet — it is a lead.

CONTENTS

Table of Contents

PART 0

Operating Doctrine

Before any payload: the discipline that keeps research lawful, safe, in-scope, and accepted on first submission. These rules outrank every technique that follows.

0.1 Reading a program correctly

A program brief is a contract. Read it completely before touching the target, and re-read it whenever the asset surprises you. Extract and record:

- **In-scope assets** — exact domains, wildcard rules, IP ranges, apps, and APIs.
- **Out-of-scope assets** — treat these as production you may never touch.
- **Out-of-scope vulnerabilities** — classes the program will close as N/A.
- **Testing constraints** — rate limits, prohibited tooling, test-account rules.
- **Safe harbor / disclosure terms** — the legal protection and its conditions.
- **PII / data-handling rules** — what you must not access, store, or exfiltrate.

> **SCOPE LEDGER**
> Maintain a per-program scope ledger (in your engagement notes) capturing the in/out lists, ROE, and the date you read them. Programs change scope; your ledger is your evidence that you were authorized at the time of test.

0.2 Rules of engagement — hard limits

- Use only program-provided or self-registered **test accounts**; never target real users.
- Demonstrate impact with the **least-invasive proof** — read one record, not the table.
- Never pivot, persist, escalate beyond proof, or move laterally into out-of-scope systems.
- No denial of service, no automated brute force against production, no destructive payloads.
- Stop on signs of real customer-data exposure; report immediately rather than exploring.

> **DESTRUCTIVE-ACTION RULE**
> If a candidate finding's proof would delete, corrupt, mass-modify, or expose third-party data, you do **not** execute it. Demonstrate the precondition (e.g., the missing authorization check) and describe the impact. The report carries the weight; the blast radius does not.

0.3 Rate, throttle, and blast-radius hygiene

Aggressive automation gets researchers banned and breaks production. Default to conservative concurrency and honor any published limits.

```
# Conservative recon defaults (adjust to the program's stated limits)
#   - cap concurrency, add jitter, respect 429/Retry-After
httpx -l hosts.txt -rl 20 -t 10 -timeout 10 -retries 1 -random-agent
ffuf -u https://TARGET/FUZZ -w words.txt -rate 30 -t 20 -ac -p 0.1-0.3
#   -rate caps requests/sec ; -p adds per-request delay (jitter)
```

Throttled defaults: low rate, jitter, automatic 429 back-off. Tune to the ROE.

0.4 Evidence & reproducibility standard

A triager accepts what they can reproduce. For every finding, capture a clean, minimal, deterministic proof chain:

- Exact request(s) and response(s) — raw HTTP, headers included, secrets redacted.
- The **two-account** or **two-session** setup when the bug is authorization-related.
- A timestamp, the asset/URL, and the test-account identifiers used.
- A short, copy-pasteable repro: cURL or a numbered click-path a triager can follow.
- Screenshots/video only as *supporting* evidence — never as the primary proof.

REDACTION DISCIPLINE

Redact session tokens, other users' PII, and program secrets in your writeup, but keep an unredacted private copy in your engagement vault in case triage needs to verify. Never paste live tokens into third-party services.

PART 1

Recon & Attack-Surface Mapping

You cannot test what you have not enumerated. This playbook turns an in-scope wildcard into a prioritized, deduplicated map of live, interesting attack surface.

1.1 Objective & scope precondition

Objective: enumerate the full authorized attack surface — subdomains, hosts, endpoints, parameters, technologies — and triage it down to the handful of targets worth deep testing.

Scope precondition: the program authorizes a wildcard (e.g. `*.target.com`) or an explicit asset list. For a wildcard, you may enumerate subdomains; you may only *test* the ones the program includes. Resolve every discovered host against the scope ledger before sending traffic.

1.2 Methodology

Step 1 — Passive subdomain enumeration (no traffic to target)

```
# Aggregate passive sources first — quiet, fast, no requests to the target
subfinder -d target.com -all -silent          > subs_passive.txt
amass enum -passive -d target.com -silent     >> subs_passive.txt
# Certificate transparency is high-signal for forgotten hosts
curl -s 'https://crt.sh/?q=%25.target.com&output=json' \
  | jq -r '.[].name_value' | sed 's/\*\.//g'   >> subs_passive.txt
sort -u subs_passive.txt -o subs_passive.txt
```

Passive-first keeps you quiet and avoids touching out-of-scope infrastructure.

Step 2 — Resolve, filter to scope, probe for live HTTP

```
# Resolve, then KEEP ONLY in-scope hosts via your scope allow-list
dnsx -l subs_passive.txt -silent -a -resp-only | sort -u > resolved.txt
grep -F -f scope_allow.txt subs_passive.txt        > in_scope.txt
# Probe live web services; record tech, status, title, CDN
httpx -l in_scope.txt -silent -sc -title -tech-detect -server \
      -rl 20 -t 10 -o live.txt
```

Filter to scope BEFORE probing. httpx fingerprints stack, status, and title in one pass.

Step 3 — Map endpoints, parameters, and historical URLs

```
# Mine archives + JS for endpoints and parameters (passive sources)
gau --subs target.com | sort -u           > urls_archive.txt
katana -u https://app.target.com -jc -d 3 -rl 20 > urls_crawl.txt
# Extract parameter names worth fuzzing later
cat urls_*.txt | unfurl keys | sort -u     > params.txt
# Pull JS and grep for routes, API bases, and leaked secrets
cat urls_*.txt | grep -Ei '\.js($|\?)' | httpx -silent -mr 'api|token'
```

Archived URLs surface deprecated endpoints; JS reveals internal API routes and keys.

Step 4 — Fingerprint and prioritize

Rank surface by likely reward. Move dev/staging/admin panels, auth endpoints, file uploaders, payment flows, and API gateways to the top of the queue. De-prioritize static marketing hosts and third-party CDNs you cannot affect.

Surface signal	Why it ranks high	Playbook to run next
staging / dev / uat host	weaker controls, debug on	Parts 2–8 broadly
/admin, /internal, SSO	high-value if broken authZ	Part 3 (AuthZ)
upload / import / avatar	RCE, SSRF, stored XSS	Parts 5, 6, 9
GraphQL / REST gateway	broad IDOR + introspection	Part 8 (API)
password reset / OAuth	account takeover chains	Part 2 (Auth)

1.3 Tooling

- **`subfinder`**, **`amass`**, **`crt.sh`** — passive subdomain sources.
- **`dnsx`**, **`httpx`** — resolution and live-host probing with fingerprints.
- **`gau`**, **`katana`**, **`waybackurls`** — endpoint and URL discovery.
- **`unfurl`**, **`gf`** — parameter extraction and pattern triage.
- A proxy (Burst/Caido/ZAP) to capture, replay, and annotate everything from here on.

1.4 Signals worth deep testing

- Hosts returning verbose errors, stack traces, or debug toolbars.
- Old API versions still live alongside a newer one (auth gaps between them).
- Subdomains pointing at unclaimed third-party services (see Part 9 — takeover).
- Parameters that echo into responses, redirects, or downstream requests.

1.5 Evidence & pitfalls

FALSE-POSITIVE TRAPS IN RECON

Wildcard DNS makes every random subdomain <anything>.target.com resolve — confirm a host is real before reporting it. Shared CDN/hosting IPs are not the program's infrastructure; do not test other tenants. A subdomain resolving is not a vulnerability — recon output alone is never a submittable finding.

Recon produces a **map**, not findings. Carry the prioritized live list into the class-specific playbooks that follow.

PART 2

Authentication & Session

Account takeover is consistently high-severity and well-rewarded. This playbook covers login, session handling, OAuth/JWT, MFA, and password-reset flows.

2.1 Objective & scope precondition

Objective: demonstrate authentication bypass, session compromise, or full/partial account takeover (ATO) using only test accounts you control.

Scope precondition: the auth surface is in scope and the program does not exclude the relevant class (some exclude self-XSS-style or rate-limit-only reports). Register at least **two** test accounts up front — ATO and authZ proofs require an attacker account and a victim account you also own.

2.2 Methodology

Step 1 — Map the auth surface

- Enumerate flows: password login, social/OAuth, magic link, SSO, API tokens, MFA.
- Capture every request/response in the proxy: login, refresh, logout, reset, MFA.
- Note where session state lives: cookie, bearer token, JWT, server session id.

Step 2 — Session management testing

- Confirm session fixation: does the session id rotate on privilege change / login?
- Check logout: is the server-side session actually invalidated, or just the cookie?
- Test cookie flags: `HttpOnly`, `Secure`, `SameSite`.
- Look for predictable, sequential, or short-entropy session identifiers.

Step 3 — JWT inspection and tampering

Decode the token, then test the implementation against the classic verification failures. Use accounts you own; never replay another user's live token.

```
# Decode header + payload (base64url) without verifying — inspection only
echo "$JWT" | cut -d. -f1 | base64 -d 2>/dev/null | jq .   # header
echo "$JWT" | cut -d. -f2 | base64 -d 2>/dev/null | jq .   # claims

# Test surfaces (against YOUR OWN token, on an in-scope endpoint):
#  - alg confusion: server accepts "none" or HS256-signed-with-pubkey
#  - signature not verified: flip a claim, resend, check for acceptance
#  - weak HMAC secret: attempt offline crack of HS256 with a wordlist
#  - kid/jku/x5u injection: does the server fetch keys from a URL you set?
#  - expiry/nbf ignored: replay a stale token, check it still authenticates
```

JWT test matrix. Each line is a distinct implementation flaw to verify, not assume.

OAUTH / JWT WRAPPER FOCUS

When an app wraps an OAuth provider's token in its own session JWT, test the seam: does the wrapper re-verify the upstream token's signature, audience (aud), and issuer (iss)? Mismatched *aud*/*iss* validation and accepting a token minted for a different client are recurring, high-impact wrapper bugs. Replay-validate with two accounts before claiming impact.

Step 4 — Password reset & pre-ATO chains

- Reset-token entropy/expiry: short, sequential, or non-expiring tokens enable ATO.
- Host-header / referer poisoning: does the reset link honor an attacker-set host?
- Response reflection: is the reset token ever returned in a response body or redirect?
- Account linking: can OAuth link to an existing email without verification (pre-ATO)?
- Email change without re-auth or confirmation, then password reset to the new mailbox.

Step 5 — MFA logic

- Can MFA be skipped by navigating directly to the post-MFA endpoint?
- Is the OTP rate-limited? Is it bound to the session that requested it?
- Does a password reset or 'remember device' silently drop the second factor?
- Backup-code reuse, OTP reuse across the validity window, or response-flag tampering.

2.3 Signals

- Token claims that map directly to identity (`uid`, `email`, `role`).
- Reset/verify tokens visible in responses, logs, or referer-leaking redirects.
- Auth decisions made client-side (a hidden field or response flag gating access).
- Different error text for valid vs invalid usernames (enumeration primitive).

2.4 Evidence & PoC

- Two-account proof: show attacker session acting as / accessing victim account.
- Full request/response chain for the bypass, with tokens redacted in the writeup.
- Deterministic repro steps; for ATO, a short video of the takeover supports the text.

PITFALLS / FALSE POSITIVES

A decodable JWT is not a vulnerability — JWTs are *designed* to be readable; only a verification failure counts. Missing cookie flags with no demonstrated impact are usually informational. Rate-limiting absence alone is low/N-A on many programs unless it enables a concrete attack. Never test ATO against a real user — only accounts you registered.

2.5 Reporting notes

Lead with impact: "full account takeover of any user via …". Map the exact precondition, the attacker capability gained, and affected user population. ATO and auth bypass typically land High–Critical; tie the CVSS vector to the demonstrated impact, not the theoretical maximum.

PART 3

Authorization Flaws

Broken object- and function-level authorization (IDOR / BOLA / BFLA) is the most common high-impact bug class in modern apps and APIs. Two accounts, one question: can account A reach account B's data or actions?

3.1 Objective & scope precondition

Objective: show that an authenticated (or anonymous) actor can read or modify resources, or invoke functions, they are not authorized for.

Scope precondition: two test accounts in different tenants/roles where possible (low-priv + second user, and ideally an admin you control). The object identifiers you manipulate must belong to **your own** second account — never another customer's real data.

3.2 Methodology

Step 1 — Inventory object references and privileged functions

- Catalog every request that takes an id: `/api/orders/1042`, `?user_id=`, UUIDs.
- Catalog privileged actions: delete, export, role-change, billing, admin endpoints.
- Note the identifier type: sequential int (easy), UUID (needs a leak), or hashed.

Step 2 — The two-account swap test

The core IDOR test: capture a request as Account B, then replay it with Account A's session (and vice-versa). If A receives B's resource, authorization is broken at the object level.

```
# Capture Account B's request, then replay with Account A's cookie/token.
# Use ONLY the ids of accounts you control.
curl -s https://app.target.com/api/orders/B_ORDER_ID \
  -H "Authorization: Bearer $TOKEN_A" -i | head -n 20
#  -> 200 + Account B's data with Account A's token == BOLA / IDOR
#  -> 403/404 == authorization enforced (expected)
```

Same object id, different identity. A 200 cross-account read is the finding.

Step 3 — Method, function, and role probing (BFLA)

- Swap the HTTP method: can a read-only user `PUT`/`DELETE` the object?
- Call admin-only endpoints with a low-priv token; check for missing function checks.
- Tamper role/tenant fields in the body (`"role":"admin"`, `"tenant_id"`).
- Mass-assignment: add privileged fields the UI never sends and see if they bind.

Step 4 — Identifier discovery (when ids are not guessable)

UUIDs and opaque ids still leak. Find them in: API list endpoints, search, exports, email notifications, referer headers, GraphQL relations, and predictable encodings (base64, sequential timestamps). A leaked id plus a

missing authZ check is a complete IDOR chain.

3.3 Signals

- Object access decided purely by the id in the request, with no ownership re-check.
- Tenant/org id present in the request body or token but not enforced server-side.
- Numeric, sequential, or trivially-encoded identifiers on sensitive resources.
- Different behavior between UI (hides the button) and API (still honors the call).

3.4 Evidence & PoC

- Side-by-side: Account A's token + Account B's object id -> B's data returned.
- Clearly label which account owns which id so triage can reproduce instantly.
- For write/delete IDOR, prove the precondition minimally (e.g. toggle one benign field).

PITFALLS / FALSE POSITIVES

A 200 that returns *your own* object after an id swap proves nothing — verify the object truly belongs to the other account. Some apps return generic public objects by id (by design). Never enumerate or read real customers' records to 'prove scale' — one controlled cross-account access is sufficient and lawful; mass enumeration is not.

3.5 Reporting notes

State the exact capability: read vs write vs delete, single-object vs enumerable, cross-tenant vs same-tenant. Enumerable cross-tenant write/delete is typically Critical; single-object same-tenant read may be Medium. Anchor severity to the demonstrated, not theoretical, reach.

PART 4

Injection

Where user input crosses into an interpreter — SQL, OS, template, NoSQL, LDAP — injection turns a parameter into code. High severity, but high false-positive risk; confirm carefully and prove safely.

4.1 Objective & scope precondition

Objective: demonstrate that attacker-controlled input is interpreted as code/query by a backend, with a safe, minimal proof of execution.

Scope precondition: the endpoint and parameter are in scope. Confirm with **non-destructive** payloads only — time-based or boolean inference, never `DROP`, `DELETE`, or data-mutating proofs on production.

4.2 Methodology

Step 1 — Find injection-reachable parameters

- Test every input that reaches a backend: query, body, JSON fields, headers, cookies.
- Prioritize search, filter, sort, id, and reporting endpoints — query-builders abound.
- Probe with safe metacharacters and watch for errors, type changes, or timing shifts.

Step 2 — SQL injection (inference-first, non-destructive)

```
# Boolean inference: TRUE vs FALSE should change the response deterministically
id=10' AND '1'='1   -- expect normal result
id=10' AND '1'='2   -- expect empty/different result  -> SQLi signal

# Time-based confirmation (blind), kept short to avoid load:
id=10';SELECT pg_sleep(3)--      (PostgreSQL)
id=10' AND SLEEP(3)-- -          (MySQL)
# Confirm a controlled, repeatable delay. Do NOT read/dump bulk data.
```

Confirm with inference and a short, controlled delay. Stop at proof-of-execution.

SAFE-PROOF RULE FOR SQLi

Proof of injection = a controlled, reproducible difference (boolean or timing) or a single harmless value like the DB version. Never extract customer data, never run destructive statements, and cap `SLEEP` durations so you do not stress the database. The missing parameterization is the bug; you do not need to exfiltrate a table to prove it.

Step 3 — Other interpreters

Class	Quick signal	Safe confirmation
Command injection	output changes on ; | `	time-based: ;sleep 3 (capped)
SSTI (templates)	{{7*7}} -> 49 in output	render arithmetic, then stop
NoSQL injection	[$ne], [$gt] alter results	boolean inference on auth/filter
LDAP injection	*)(&) changes auth result	boolean inference only
XXE (XML input)	entity expands / errors	OOB or error-based; see Part 9

For SSTI, escalate only as far as proving template evaluation (**`{{7*7}}`**→49). Stop short of OS-command payloads on production; describe the RCE path in the report rather than executing it.

4.3 Tooling

- A proxy + manual payloads first — understand the parser before automating.
- **`sqlmap`** with **`--technique`** limited and **`--safe-url`**, low threads, only after manual signal.
- **`tplmap`**-style SSTI helpers for fingerprinting the engine (verify manually).
- Custom small wordlists per context; broad fuzzing is noisy and ban-prone.

4.4 Evidence, pitfalls & reporting

- Show the exact parameter, the inference pair (true/false) or timing delta, and repro.
- Identify the interpreter and the missing control (parameterization, sandboxing).
- WAF-reflected payloads and self-inflicted errors are not injection — confirm execution.

FALSE POSITIVES

A reflected error message is not SQLi. A **`{{7*7}}`** that prints literally is not SSTI. Response timing can vary from network noise — require a *repeatable* delta across several trials before claiming blind injection.

Injection with confirmed execution is typically High–Critical. Report the class, the proven capability, and the safe proof — explicitly note you withheld destructive exploitation by policy.

PART 5

Cross-Site Scripting (XSS)

Reflected, stored, and DOM-based XSS remain bread-and-butter findings. The skill is in proving real impact in a real context — not popping an alert in a sink that harms no one.

5.1 Objective & scope precondition

Objective: execute attacker-controlled script in another user's authenticated browser context, in scope, with demonstrable impact.

Scope precondition: the sink is in scope and the program does not exclude self-XSS. Prove impact with your own test victim; never deliver a payload to real users.

5.2 Methodology

Step 1 — Identify reflection/storage and its context

- Inject a unique marker; find where it lands: HTML body, attribute, JS, URL, or DOM.
- Context determines the payload — HTML vs attribute vs JS string vs URL each differ.
- For stored XSS, find inputs rendered to *other* users (profiles, comments, tickets).

Step 2 — Break out of the context (minimal, harmless proof)

```
# Context-appropriate break-out; prove execution, not damage.
# HTML body sink:
  <img src=x onerror=alert(document.domain)>
# Attribute sink (close the attribute/tag first):
  "><svg onload=alert(document.domain)>
# JS-string sink:
  ';alert(document.domain)//
# Prefer document.domain over alert(1): it proves WHERE the code runs.
```

Use document.domain so the screenshot proves the origin of execution.

Step 3 — DOM XSS hunting

- Trace sources (`location`, `document.referrer`, `postMessage`) to sinks (`innerHTML`, `eval`).
- Test client-side routing/hash params and templating that writes raw HTML.
- Use the proxy + browser devtools; `DOMPurify` gaps and sink misuse are common.

5.3 Impact, evidence & pitfalls

- Show the rendered execution with `document.domain` and the exact injection point.
- For stored XSS, demonstrate it firing in a *second* account's session you control.
- State realistic impact: session theft, action-on-behalf, credential capture in-context.

FALSE POSITIVES / DOWNGRADES

Self-XSS (only the victim can inject into their own page) is low/N-A on most programs account, no-data static page is low impact. An `alert(1)` with no path to affecting another user is weak — always tie the finding to a concrete victim impact. Never mass-deliver a payload to prove reach.

Reflected XSS is often Medium; stored XSS hitting other users or admins is High. CSP, HttpOnly cookies, and framing protections can reduce real impact — account for them honestly in your severity.

PART 6

Server-Side Request Forgery

SSRF turns the application into your proxy into internal networks and cloud metadata. High severity in cloud environments — and high responsibility, because the blast radius is internal infrastructure.

6.1 Objective & scope precondition

Objective: make the server issue attacker-controlled requests to internal or unintended destinations, proven minimally via out-of-band (OOB) interaction or a controlled internal response.

Scope precondition: SSRF is in scope. Do **not** pivot into internal systems, read live cloud credentials, or interact with anything beyond the minimum needed to prove the server made the request. Cloud-metadata and internal-port access is the proof ceiling unless the program explicitly invites more.

6.2 Methodology

Step 1 — Find request-issuing features

- URL fetchers: webhooks, link previews, PDF/HTML render, import-from-URL, avatars.
- Anything that takes a URL, hostname, or file path the server then retrieves.
- XML/SVG/Office parsers (SSRF via XXE/external refs) and image-processing pipelines.

Step 2 — Confirm with out-of-band interaction

```
# Point the fetch at YOUR collaborator host and watch for the callback.
# Confirms the server made the request without touching internal systems.
url=https://YOUR-OOB-ID.oast.site/ssrf-check
#  -> DNS + HTTP hit on your listener == SSRF confirmed (server-side)

# Then test, minimally, whether internal targets are reachable:
url=http://169.254.169.254/        (cloud metadata endpoint — STOP at proof)
url=http://127.0.0.1:PORT/         (internal port reachability)
```

OOB callback first — it proves SSRF cleanly without entering internal networks.

BLAST-RADIUS CEILING

Proving the server fetched your OOB host, or that the metadata endpoint is reachable, is sufficient. Do **not** retrieve live IAM credentials, enumerate internal services, or pivot — that exceeds authorization and risks real harm. Describe the credential-theft / internal-access impact in the report; do not execute it.

Step 3 — Defeat naive filters (only to prove the bypass)

- Alternate IP encodings, `[::]`, DNS rebinding, and `@`-in-URL tricks vs allow-lists.
- Redirect-based SSRF: server follows a 302 from your URL to an internal target.
- Protocol smuggling (`gopher:`, `file:`, `dict:`) where the fetcher allows it.

6.3 Evidence, pitfalls & reporting

- Primary proof: the OOB callback log (timestamp, source IP) tied to your request.
- If you reached metadata/internal, show the minimal response that proves reachability.
- State the realistic escalation (cloud creds, internal API access) without performing it.

FALSE POSITIVES

A client-side request, or a request your own browser made, is not SSRF — the *server* must originate it. A fetch that resolves only to public, intended destinations is not a vulnerability. Confirm the callback came from server infrastructure, not your own host.

Internal/metadata-reachable SSRF in cloud environments is typically High–Critical. Severity tracks what the server could reach, evidenced by your minimal proof — not speculation.

PART 7

Business Logic & Race Conditions

No scanner finds these. Logic flaws live in the gap between what the developer assumed and what the workflow actually permits — discounts, limits, state transitions, and concurrency.

7.1 Objective & scope precondition

Objective: abuse the intended workflow to reach an unintended, advantageous state — free goods, bypassed limits, skipped steps, value creation.

Scope precondition: use test accounts and, where money is involved, test/sandbox payment instruments only. Never complete a real fraudulent transaction; prove the flaw at the point the logic fails.

7.2 Methodology

Step 1 — Model the intended workflow, then break each assumption

- Diagram the multi-step flow (cart → pay → fulfill; apply → approve → grant).
- For each step ask: can I skip it, repeat it, reorder it, or reach it out of sequence?
- Tamper quantities, prices, currencies, coupon stacking, and negative/overflow values.
- Check server-side enforcement of every limit the UI implies (max qty, one-per-user).

Step 2 — State-machine and step-skip testing

- Jump directly to a later-stage endpoint without completing prerequisites.
- Replay a 'completed' transition; re-trigger fulfillment or re-grant entitlements.
- Downgrade/upgrade flows: keep premium features after reverting to a free plan.

Step 3 — Race conditions (limit bypass via concurrency)

When a check and an action are not atomic, parallel requests can both pass the check before either commits — redeeming a one-time coupon twice, over-withdrawing, double-spending an entitlement.

```
# Single-packet / parallel send to exploit the check-then-act gap.
# Use small, controlled concurrency on a test resource you own.
#   Burp Repeater 'send group in parallel', or Turbo Intruder, or:
seq 20 | xargs -P20 -I_ curl -s -X POST \
  https://app.target.com/api/redeem -H "Authorization: Bearer $T" \
  -d '{"code":"YOUR_ONE_TIME_CODE"}' -o /dev/null -w '%{http_code}\n'
#  Multiple successes for a single-use code == race condition.
```

Keep concurrency modest and the target a resource you own. Confirm, then stop.

MONEY & HARM BOUNDARY

Logic and race testing can move real money or inventory. Use sandbox payment methods and your own accounts; prove the flaw with the smallest possible effect and immediately stop. If a flaw would cause real financial loss to the program or third parties, demonstrate the precondition and report — do not realize the loss.

7.3 Evidence, pitfalls & reporting

- Show the before/after state proving the unintended outcome (e.g. coupon used twice).
- Capture the parallel requests and their successful responses for race findings.
- Quantify business impact concretely (revenue loss path, limit bypassed, value created).

FALSE POSITIVES

A client-side-only limit that the server re-enforces is not a bug. A 'discount' that the backend recomputes correctly is not exploitable. Race 'successes' that the system later reconciles/rolls back are not real — verify the advantageous state persists.

Logic flaws are judged on business impact, so make the impact explicit and reproducible. High-value financial logic bugs frequently rate High–Critical despite needing no exotic payload.

PART 8

API Testing (REST & GraphQL)

APIs concentrate the highest-yield bugs: authorization gaps, mass assignment, and over-exposure. The UI is a courtesy; the API is the real attack surface.

8.1 Objective & scope precondition

Objective: find authorization, exposure, and input-handling flaws at the API layer — frequently more permissive than the UI implies.

Scope precondition: the API hosts/endpoints are in scope. Authenticate with your own tokens; object ids you manipulate belong to accounts you control.

8.2 Methodology — REST

- Recover the API contract: Swagger/OpenAPI, `/docs`, JS API clients, old versions.
- Run the Part 3 authZ swap on every object endpoint (BOLA is the #1 API bug).
- Test function-level authZ (BFLA): low-priv token against admin endpoints/methods.
- Mass assignment: add fields the client never sends (`isAdmin`, `verified`).
- Excessive data exposure: does the response include fields the UI hides (PII, hashes)?
- Version drift: `/v1` may lack a fix present in `/v2` — test both.

8.3 Methodology — GraphQL

```
# 1) Try introspection — it maps the entire schema if left enabled.
query{__schema{types{name fields{name}}}}

# 2) Hunt authZ per-resolver: a guarded query may have an unguarded sibling.
# 3) Batching/alias abuse to amplify or bypass per-request rate limits:
query{a:user(id:1){email} b:user(id:2){email} c:user(id:3){email}}
# 4) Deeply nested queries to probe DoS-style resource exhaustion (be gentle).
```

GraphQL widens surface: introspection, per-resolver authZ, alias batching, depth.

- Even with introspection disabled, infer types via field suggestions and error text.
- Mutations often lack the authZ rigor of the UI path — test create/update/delete directly.
- Alias batching can defeat naive per-request throttling and enable enumeration.

8.4 Evidence, pitfalls & reporting

- For BOLA/BFLA, the two-account swap proof from Part 3 applies verbatim.
- For exposure, show the exact over-returned fields (redact real PII in the writeup).
- Note auth context precisely: anonymous vs low-priv vs cross-tenant.

FALSE POSITIVES

Introspection being enabled is usually *informational* on its own — pair it with a concrete authZ or exposure bug for impact. An endpoint returning your own extra fields is not exposure; it must reveal data you should not see. Don't enumerate real users to show scale — one controlled cross-account proof is enough.

API authorization flaws inherit the severity of the data/actions they unlock — frequently High–Critical when cross-tenant or admin-reaching. Be precise about the auth context in the CVSS vector.

PART 9

High-Value Supplementary Classes

Compact playbooks for classes that round out a hunt: subdomain takeover, file upload, XXE, CSRF, and open redirect. Same discipline, condensed form.

9.1 Subdomain takeover

Signal: an in-scope subdomain CNAMEs to a third-party service with no active claim (dangling DNS). **Proof ceiling:** claim the resource and serve a unique, benign marker page — never host real content or abuse the domain.

```
# Detect dangling CNAMEs against known fingerprints
subzy run --targets in_scope.txt --concurrency 10
# Manual check: does the CNAME target return a 'NoSuchBucket'/'not found' claim page?
dig +short cname stale.target.com
#  -> if claimable, register it and serve only a harmless PoC marker, then report.
```

Claim + benign marker is sufficient proof. Release the resource after reporting.

TAKEOVER DISCIPLINE

Serve only a unique text marker tied to your report id. Do not collect traffic, set cookies, or run any content against visitors. Relinquish the claimed resource once the report is filed so it cannot be abused.

9.2 Unrestricted file upload

- Test content-type, extension, and magic-byte validation independently — each may differ.
- Probe for stored XSS (SVG/HTML), SSRF (XML/SVG external refs), and path traversal in names.
- For RCE-via-upload, prove the upload + reachable path; **do not** execute a web shell on prod.
- Image-parser exploits: confirm the parser is reachable; describe impact rather than weaponize.

9.3 XML external entity (XXE)

```
# OOB XXE confirmation — proves external entity processing without reading files.
<?xml version="1.0"?>
<!DOCTYPE r [ <!ENTITY x SYSTEM "https://YOUR-OOB-ID.oast.site/xxe"> ]>
<r>&x;</r>
#  Callback on your listener == XXE. Stop at proof; do not read sensitive files.
```

Confirm via OOB callback. File-read/SSRF escalation is described, not performed.

- Find XML sinks: SOAP, SAML, SVG/Office uploads, `application/xml` bodies, RSS import.
- Blind XXE: use an external DTD + OOB exfil channel to confirm; cap what you retrieve.

9.4 CSRF

- Target state-changing requests lacking anti-CSRF tokens or **`SameSite`** protection.
- Verify the action truly executes cross-origin with only ambient cookies (no token echo).
- Build a minimal self-submitting PoC page against **your own** test account.
- Login/logout CSRF and OAuth-state CSRF are commonly overlooked, sometimes chainable.

> **CSRF IMPACT TEST**
> A request without a CSRF token is only a finding if it (a) changes state and (b) actually succeeds cross-origin. If **`SameSite=Lax/Strict`** or a verified token blocks it, there is no CSRF — confirm execution from a foreign origin before reporting.

9.5 Open redirect

- Test redirect/return/next params for off-domain navigation the app should reject.
- Low severity alone; report the **chain** — OAuth token theft, SSRF pivot, phishing trust.
- Bypass naive checks with **`//evil.com`**, **`\evil.com`**, and `@`-host tricks (to prove the gap).

PART 10

Reporting & Triage

A great bug with a weak report gets closed. The writeup is the deliverable — make impact obvious, reproduction trivial, and severity defensible.

10.1 Anatomy of a report that gets paid

Section	What it must contain
Title	Impact-first: '<impact> via <flaw> on <asset>'
Summary	2–3 sentences: what, where, who is affected, why it matters
Affected asset	Exact in-scope URL/endpoint + the test accounts used
Steps to reproduce	Numbered, deterministic, copy-pasteable; no missing setup
Proof / PoC	Raw requests/responses (redacted), minimal cURL, supporting media
Impact	Concrete attacker capability + affected population; realistic, not max
Severity	CVSS 3.1/4.0 vector with justification tied to the demonstrated impact
Remediation	Specific, actionable fix (the missing control, not 'sanitize input')

10.2 Severity, honestly

Anchor CVSS to what you **demonstrated**, not the theoretical ceiling. Account for compensating controls (CSP, `SameSite`, MFA, tenant isolation). Over-claiming severity erodes triager trust and costs you on future reports.

Tier	Typical examples (impact-dependent)
Critical	Pre-auth RCE, full ATO at scale, cross-tenant mass data access
High	Authenticated RCE, stored XSS hitting admins, broad IDOR, SSRF to cloud creds
Medium	Reflected XSS, single-object IDOR, CSRF on sensitive action, SSRF (limited)
Low	Open redirect, verbose errors, self-XSS chains, minor info disclosure
Info	Best-practice gaps with no demonstrated impact (note, don't over-rate)

10.3 Duplicates, collisions & disclosure

- Report promptly and completely — first valid, reproducible report usually wins.
- Accept duplicate rulings gracefully; reputation compounds across a program.
- Honor coordinated-disclosure timelines; never disclose publicly before authorized.
- Keep your private evidence vault until the report is resolved and disclosure agreed.

GREYNOC SUBMISSION CHECKLIST

Reproduced twice from a clean session · only test accounts / sandbox payments used · stayed within scope & ROE · least-invasive proof · secrets and third-party PII redacted · impact stated concretely · CVSS justified · remediation specific · no fabrication.

APPENDIX A

Operator Quick-Reference Card

A.1 Recon one-liners

```
subfinder -d target.com -all -silent | dnsx -silent | \
  grep -F -f scope_allow.txt | httpx -silent -sc -title -tech-detect -rl 20
gau --subs target.com | sort -u | gf xss; gau --subs target.com | gf ssrf
katana -u https://app.target.com -jc -d 3 -rl 20 | unfurl keys | sort -u
```

Scope-filtered recon → live hosts → pattern-tagged URLs. Mind the rate limits.

A.2 Class-to-test cheat sheet

Class	First move	Safe proof
IDOR / BOLA	two-account id swap	A's token reads B's object (you own both)
JWT	decode header/claims	alg=none / unverified-sig acceptance
SQLi	boolean + time inference	repeatable true/false or capped delay
XSS	marker → find context	document.domain executes in victim ctx
SSRF	OOB callback URL	server hits your listener; metadata reachable
Race	parallel send (modest)	single-use action succeeds N times
Takeover	dangling CNAME scan	claim + unique benign marker, then release

ALWAYS

Re-check the scope ledger before sending traffic · throttle to the ROE · use accounts and ids you control · take the least-invasive proof · stop at confirmation · report, don't exploit further. Authorized testing only. Reproducible findings. No fabrication.